SQL 종합실습 3

튜닝 전·후 실행 계획 및 성능 비교 보고서

작성자 : 최승우

2026-08-13

0. 실습 과제

1. 리포트로 느린 이유, 그에 따른 개선 방법 도출, 개선 후 주요효과에 대해 실행 계획 이전/ 이후 결과 정리를 자세히 할 것

2. 최소 2개 이상 쿼리를 만들고 쿼리에 대한 차이점을 분석 결과를 작성할것

1. 사번이 100인 사람 검색

쿼리문

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM employees
WHERE employee_id = 100;
```

성능

```
 Index Scan using employees_pkey on employees
(cost=0.29..8.31 rows=1 width=94) (actual time=0.036..0.036 rows=1 loops=1)
   Index Cond: (employee_id = 100)
   Buffers: shared hit=3
 Planning Time: 0.110 ms
 Execution Time: 0.052 ms
(5 rows)
```

2. 인덱스 없는 함수 조건 쿼리 튜닝

1. 문제

OR 조건 쿼리 작성 후 쿼리 튜닝

(조건 : 부서코드가 10 또는 직무가 3, 4, 5 안에 있는 사람 검색)

2. 튜닝 전 쿼리 및 출력결과

튜닝 전 테스트 2-1

쿼리문 (모두 동일)

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM hr.employees
WHERE lower(email) = 'user1234@corp.com';
```

출력결과

```
Seq Scan on employees (actual time=4.816..4.818 rows=0)
  Filter: lower(email) = 'user1234@corp.com'
  Rows Removed by Filter: 50000
  Buffers: shared hit=786
Planning Time: 0.065 ms
Execution Time: 4.834 ms
```

튜닝 전 테스트 2-2

쿼리문 (승우)

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM hr.employees
WHERE lower(status) = 'inactive';
```

쿼리문 (민주, 혜민)

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT employee_id, first_name, last_name, email
FROM hr.employees
WHERE lower(email) = 'user1234@corp.com';
```

쿼리문 (형준, 의진)

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM hr.employees
WHERE lower(email) = lower('USER100@CORP.COM');
```

출력결과

```
Seq Scan on employees (actual time=0.002..4.017 rows=2522)
  Filter: lower(status) = 'inactive'
  Rows Removed by Filter: 47478
  Buffers: shared hit=786
Planning Time: 0.011 ms
Execution Time: 4.059 ms
```

3. 튜닝 작업 내역

```
CREATE INDEX idx_employees_lower_email
ON hr.employees (lower(email));

CREATE INDEX idx_employees_status
ON hr.employees (status);
ANALYZE hr.employees;
```

4. 튜닝 후 쿼리 및 출력결과

튜닝 후 테스트 **2-1**

쿼리문

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM hr.employees
WHERE lower(email) = 'user1234@corp.com';
```

출력결과

```
Index Scan using idx_employees_lower_email (actual time=0.010..0.010 rows=0)
  Index Cond: lower(email) = 'user1234@corp.com'
  Buffers: shared read=3
Planning Time: 0.066 ms
Execution Time: 0.016 ms
```

튜닝 후 테스트 **2-2**

쿼리문

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM hr.employees
WHERE status = 'INACTIVE';
```

출력결과

```
Index Scan using idx_employees_status (actual time=0.010..0.387 rows=2522)
  Index Cond: status = 'INACTIVE'
  Buffers: shared hit=757 read=4
Planning Time: 0.009 ms
Execution Time: 0.430 ms
```

5. 성능 비교

테스트	튜닝 전	전 시간	튜닝 후	후 시간	개선율
2-1 lower(email)	Seq Scan	4.834 ms	Index Scan	0.016 ms	**99.7%**
2-2 status	Seq Scan	4.059 ms	Index Scan	0.430 ms	**89.4%**

6. 추가 포인트 및 조별 의견

email은 대소문자 무시 검색이 필요하므로 lower(email) 표현식 인덱스가 적합하다. status는 값이 이미 대문자로 정규화되어 있어 컬럼에서 lower()를 제거하는 편이 낫다. ACTIVE는 95%라 인덱스 이점이 적고, 5%인 INACTIVE 조회로 비교해야 효과가 명확하다.

7. 결론 및 최적의 튜닝 **Point**

이메일은 lower(email) 표현식 인덱스, status는 입력값을 정규화하고 status = 'INACTIVE'로 직접 비교한다.

3. LIKE 접미사 검색 쿼리 튜닝

1. 문제

LIKE 검색 쿼리 작성 후 쿼리 튜닝 (대상 : '%gmail.com' 같은 접미사 검색)

2. 튜닝 전 쿼리 및 출력결과

튜닝 전 테스트 3-1

쿼리문

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM hr.employees
WHERE email LIKE '%@gmail.com';
```

출력결과

```
[Seq Scan on employees  (cost=0.00..1411.00 rows=1515 width=94) (actual time=0.012..10.146 rows=1438 loops=1)
   Filter: (email ~~ '%@gmail.com'::text)
   Rows Removed by Filter: 48562
   Buffers: shared hit=786
 Planning:
   Buffers: shared hit=94 dirtied=10
 Planning Time: 0.788 ms
 Execution Time: 10.219 ms
(8 rows)
```

튜닝 전 테스트 3-2

쿼리문

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT employee_id, email FROM hr.employees
WHERE email LIKE '%@outlook.com';
```

출력결과

```
 Seq Scan on employees  (cost=0.00..1411.00 rows=5556 width=28) (actual time=0.017..9.616 rows=5953 loops=1)
   Filter: (email ~~ '%@outlook.com'::text)
   Rows Removed by Filter: 44047
   Buffers: shared hit=786
 Planning Time: 0.119 ms
 Execution Time: 9.911 ms
(6 rows)
```

3. 튜닝 작업 내역

```
CREATE INDEX idx_employees_reverse_email
ON hr.employees (reverse(email) text_pattern_ops);

ANALYZE hr.employees;
```

4. 튜닝 후 쿼리 및 출력결과

튜닝 후 테스트 3-1

쿼리문 (모두 동일)

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM hr.employees
WHERE reverse(email) LIKE reverse('@gmail.com') || '%';
```

출력결과

```
[ Bitmap Heap Scan on employees  (cost=47.64..866.22 rows=1515 width=94) (actual time=0.508..2.329 rows=1438 loops=1)
   Filter: (reverse(email) ~~ 'moc.liamg@%'::text)
   Heap Blocks: exact=655
   Buffers: shared hit=665
   ->  Bitmap Index Scan on idx_employees_reverse_email  (cost=0.00..47.27 rows=1485 width=0) (actual time=0.336..0.337
rows=1438 loops=1)
         Index Cond: ((reverse(email) ~>=~ 'moc.liamg@'::text) AND (reverse(email) ~<~ 'moc.liamgA'::text))
         Buffers: shared hit=10
 Planning:
   Buffers: shared hit=3 dirtied=1
 Planning Time: 0.271 ms
 Execution Time: 2.452 ms
(11 rows)
```

튜닝 후 테스트 3-2

쿼리문

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT employee_id, email FROM hr.employees
WHERE reverse(email) LIKE reverse('@outlook.com') || '%';
```

출력결과

```
 Bitmap Heap Scan on employees  (cost=181.34..1056.46 rows=6061 width=28) (actual time=0.837..6.029 rows=5953 loops=1)
   Filter: (reverse(email) ~~ 'moc.kooltuo@%'::text)
   Heap Blocks: exact=785
   Buffers: shared hit=817
   ->  Bitmap Index Scan on idx_employees_reverse_email  (cost=0.00..179.82 rows=5941 width=0) (actual time=0.709..0.709
rows=5953 loops=1)
         Index Cond: ((reverse(email) ~>=~ 'moc.kooltuo@'::text) AND (reverse(email) ~<~ 'moc.kooltuoA'::text))
         Buffers: shared hit=32
 Planning Time: 0.204 ms
 Execution Time: 6.388 ms
(9 rows)
```

쿼리문 (`pg_trgm` + GIN)

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_emp_email_trgm ON hr.employees USING gin (email gin_trgm_ops);
ANALYZE hr.employees;
```

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM hr.employees WHERE email LIKE '%gmail.com';
```

5. 추가 포인트 및 조별 의견

- 접미사가 있는 경우 reverse를 통해서 문자열부터 조회하게 하여 속도 향상을 확인했다.
- 접두사 검색이 빠른 이유는 '시작점 주소 저장' 때문이 아니라, B-Tree 인덱스가 문자열의 첫 글자부터 사전순으로 정렬되어 특정 구간만 바로 탐색할 수 있기 때문입니다.
- 접미사 검색은 끝 글자를 기준으로 찾기 때문에 정렬 특성을 활용하지 못해 전체 데이터를 모두 뒤져야**(Full Scan)** 하므로 속도가 느립니다.

6. 결론 및 최적의 튜닝 **Point**

- 접미사 검색의 경우, Reverse 문자열 검색을 하면 비트맵 스캔으로 쿼리 속도가 빨라진다.

7. 추가 인덱스 이외의 튜닝 실제 적용

조건식 재작성**: split_part()**을 통해 순차 검색한다**.**

실행 쿼리

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM hr.employees
WHERE split_part(email, '@', 2) = 'gmail.com';

EXPLAIN (ANALYZE, BUFFERS)
SELECT employee_id, first_name, last_name, email
FROM hr.employees
WHERE split_part(email, '@', 2) = 'outlook.com';
```

실측 결과 및 판단

```
                                   QUERY PLAN
-------------------------------------------------------------------------------------------------------------
 Seq Scan on employees  (cost=0.00..1536.00 rows=250 width=94) (actual time=0.022..9.382 rows=1438 loops=1)
   Filter: (split_part(email, '@'::text, 2) = 'gmail.com'::text)
   Rows Removed by Filter: 48562
   Buffers: shared hit=786
 Planning Time: 0.089 ms
 Execution Time: 9.452 ms
(6 rows)

                                   QUERY PLAN
-------------------------------------------------------------------------------------------------------------
 Seq Scan on employees  (cost=0.00..1536.00 rows=250 width=49) (actual time=0.010..7.012 rows=5953 loops=1)
   Filter: (split_part(email, '@'::text, 2) = 'outlook.com'::text)
   Rows Removed by Filter: 44047
   Buffers: shared hit=786
 Planning Time: 0.059 ms
 Execution Time: 7.185 ms
(6 rows)
```

최종 튜닝 Point

> 접미사 검색은 쿼리 재작성만으로 개선되지 않았다. 반복 검색이라면 도메인 컬럼 정규화 또는 reverse(email) 표현식 인덱스를 검토한다.

4. ORDER BY와 필터 결합 쿼리 튜닝

1. 문제

최근 365일 이내에 입사한 ACTIVE 직원을 연봉 내림차순으로 100명만 반환할 때 전체 스캔과 top-N 정렬을 제거한다.

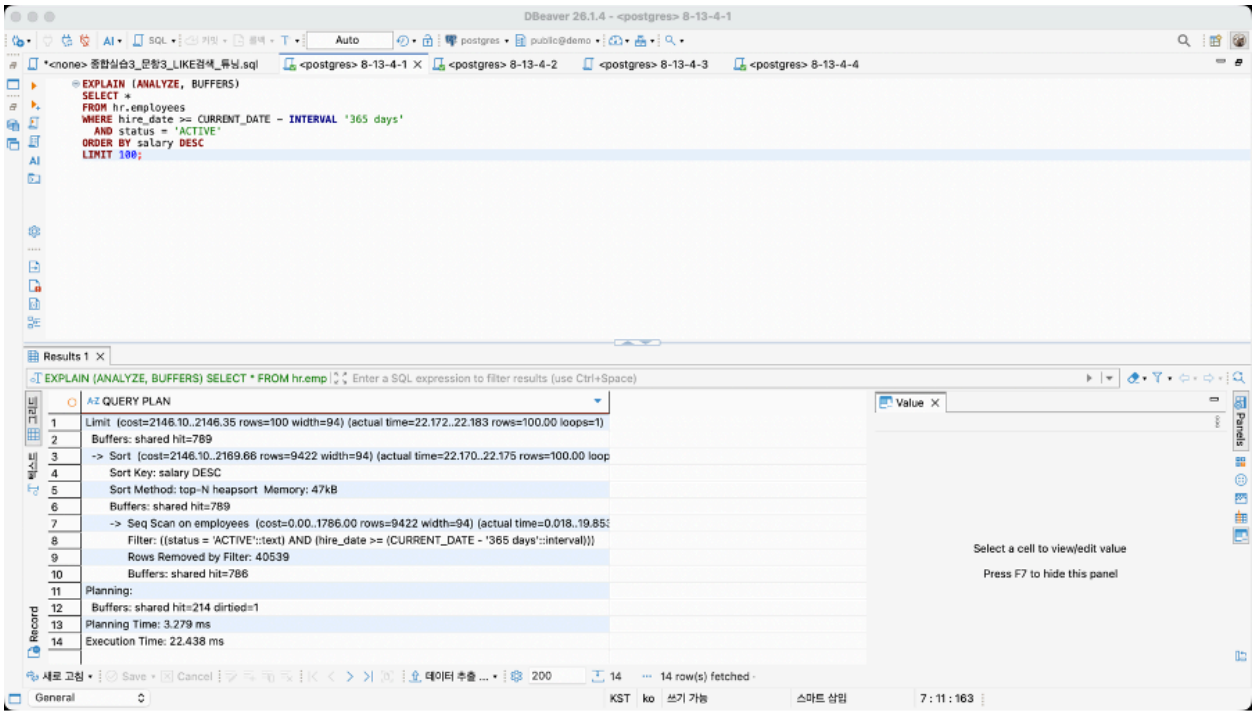
2. 튜닝 전 쿼리 및 출력결과

튜닝 전 테스트 4-1

쿼리문 (모두 동일)

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM hr.employees
WHERE hire_date >= CURRENT_DATE - INTERVAL '365 days'
  AND status = 'ACTIVE'
ORDER BY salary DESC LIMIT 100;
```

출력결과



튜닝 전 테스트 4-2

쿼리문

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT employee_id, email, hire_date, salary
FROM hr.employees
WHERE status = 'ACTIVE'
  AND hire_date >= CURRENT_DATE - INTERVAL '365 days'
ORDER BY salary DESC LIMIT 100;
```

출력결과

```
                                                              QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=0.29..48.33 rows=100 width=36) (actual time=0.030..0.811 rows=100 loops=1)
   Buffers: shared hit=483
   ->  Index Scan using idx_employees_active_salary on employees  (cost=0.29..4500.54 rows=9367 width=36) (actual time=0.028..0.801 rows=100 loops=1)
         Filter: (hire_date >= (CURRENT_DATE - '365 days'::interval))
         Rows Removed by Filter: 383
         Buffers: shared hit=483
 Planning Time: 0.197 ms
 Execution Time: 0.832 ms
(8 rows)
```

3. 튜닝 작업 내역

```
CREATE INDEX idx_employees_active_salary
ON hr.employees (salary DESC)
WHERE status = 'ACTIVE';

ANALYZE hr.employees;
```

4. 튜닝 후 쿼리 및 출력결과

주현 버전

```sql
CREATE INDEX idx_emp_active_salary_top
ON hr.employees (salary DESC, employee_id)
INCLUDE (hire_date)
WHERE status = 'ACTIVE';
```

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    employee_id,
    first_name,
    last_name,
    hire_date,
    salary,
    status
FROM hr.employees
WHERE hire_date >= (CURRENT_DATE - INTERVAL '365 days')::date
  AND status = 'ACTIVE'
ORDER BY salary DESC, employee_id
LIMIT 100;
```

esults 1 ×

XPLAIN (ANALYZE, BUFFERS) SELECT employee_id, first_name, Enter a SQL expression to filter results (use Ctrl+Space)

	QUERY PLAN
1	Limit (cost=0.41..56.68 rows=100 width=44) (actual time=0.048..0.580 rows=100 loops=1)
2	Buffers: shared hit=440 read=5
3	-> Index Scan using idx_emp_active_salary_top on employees (cost=0.41..5281.96 rows=9387 width=44) (actual time=0.047..0.573 rows=100 loops=1)
4	Filter: (hire_date >= ((CURRENT_DATE - '365 days'::interval))::date)
5	Rows Removed by Filter: 341
6	Buffers: shared hit=440 read=5
7	Planning:
8	Buffers: shared hit=22 read=1
9	Planning Time: 0.384 ms
10	Execution Time: 0.594 ms

민주 버전

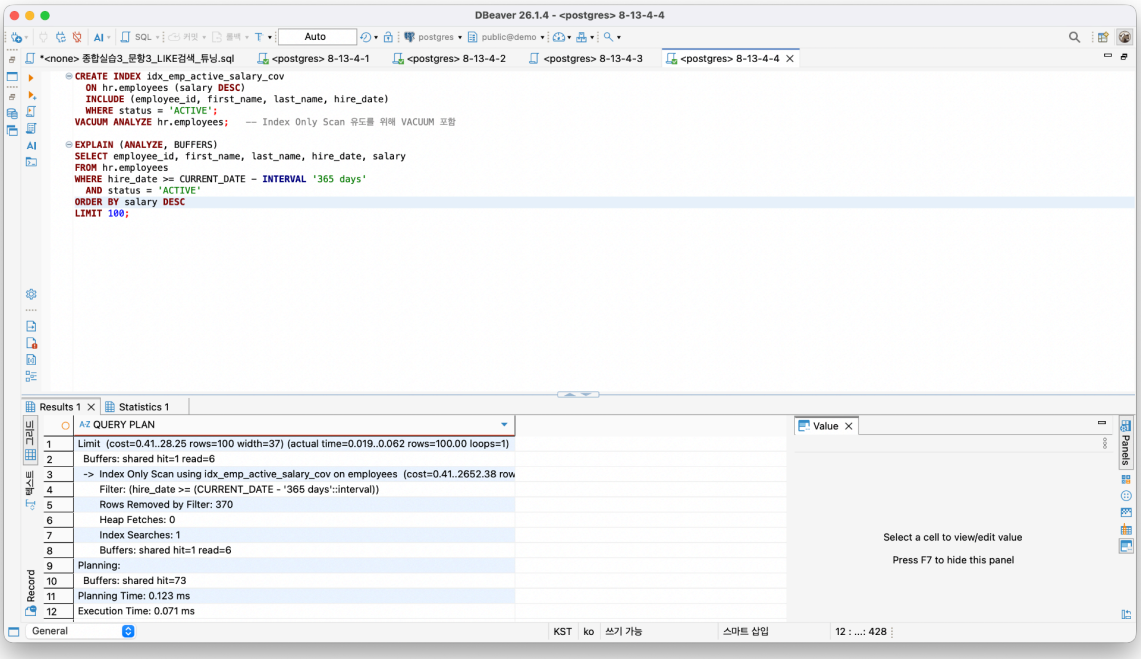
```sql
-- ACTIVE 직원 중 최근 1년 입사자 검색에 맞춘 부분 인덱스
CREATE INDEX IF NOT EXISTS idx_emp_active_hire_salary
ON hr.employees (hire_date, salary DESC)
WHERE status = 'ACTIVE';

-- 통계 갱신
ANALYZE hr.employees;

-- 튜닝 후 실행
EXPLAIN (ANALYZE, BUFFERS)
SELECT employee_id, first_name, last_name, hire_date, salary, status
FROM hr.employees
WHERE status = 'ACTIVE'
  AND hire_date >= CURRENT_DATE - INTERVAL '365 days'
ORDER BY salary DESC
LIMIT 100;
```

Results 1

EXPLAIN (ANALYZE, BUFFERS) SELE | Enter a SQL expression to filter results (use Ctrl+Space)

	QUERY PLAN
1	Limit (cost=1505.19..1505.44 rows=100 width=44) (actual time=5.870..5.881 rows=100 loops=1)
2	Buffers: shared hit=814
3	-> Sort (cost=1505.19..1528.50 rows=9321 width=44) (actual time=5.868..5.873 rows=100 loops=1)
4	Sort Key: salary DESC
5	Sort Method: top-N heapsort Memory: 36kB
6	Buffers: shared hit=814
7	-> Bitmap Heap Scan on employees (cost=176.53..1148.95 rows=9321 width=44) (actual time=1.082..4.497 rows=9486 loops=1)
8	Recheck Cond: ((hire_date >= (CURRENT_DATE - '365 days'::interval)) AND (status = 'ACTIVE'::text))
9	Heap Blocks: exact=786
10	Buffers: shared hit=814
11	-> Bitmap Index Scan on idx_emp_active_hire_salary (cost=0.00..174.20 rows=9321 width=0) (actual time=0.999..0.999 rows=9486 loops=1)
12	Index Cond: (hire_date >= (CURRENT_DATE - '365 days'::interval))
13	Buffers: shared hit=28
14	Planning:
15	Buffers: shared hit=31
16	Planning Time: 0.445 ms
17	Execution Time: 5.907 ms

형준 버전



튜닝 후 테스트 **4-1**

쿼리문

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM hr.employees
WHERE hire_date >= CURRENT_DATE - INTERVAL '365 days'
  AND status = 'ACTIVE'
ORDER BY salary DESC LIMIT 100;
```

출력결과

```
                                                  QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=0.29..48.35 rows=100 width=94) (actual time=0.016..0.716 rows=100 loops=1)
   Buffers: shared hit=483
   ->  Index Scan using idx_employees_active_salary on employees  (cost=0.29..4501.82 rows=9367 width=94) (actual
time=0.015..0.704 rows=100 loops=1)
         Filter: (hire_date >= (CURRENT_DATE - '365 days'::interval))
         Rows Removed by Filter: 383
         Buffers: shared hit=483
 Planning:
   Buffers: shared hit=46
 Planning Time: 0.484 ms
 Execution Time: 0.736 ms
(10 rows)
```

튜닝 후 테스트 **4-2**

쿼리문

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT employee_id, email, hire_date, salary
FROM hr.employees
WHERE status = 'ACTIVE'
  AND hire_date >= CURRENT_DATE - INTERVAL '365 days'
ORDER BY salary DESC LIMIT 100;
```

출력결과

```
                                                  QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=0.29..48.35 rows=100 width=36) (actual time=0.026..0.697 rows=100 loops=1)
   Buffers: shared hit=483
   ->  Index Scan using idx_employees_active_salary on employees  (cost=0.29..4501.82 rows=9367 width=36) (actual
time=0.025..0.691 rows=100 loops=1)
         Filter: (hire_date >= (CURRENT_DATE - '365 days'::interval))
         Rows Removed by Filter: 383
         Buffers: shared hit=483
 Planning Time: 0.164 ms
 Execution Time: 0.714 ms
(8 rows)
```

쿼리문 (부분/커버링 인덱스를 통한 테이블 접근 차단)

```
CREATE INDEX idx_emp_active_salary_cov
ON hr.employees (salary DESC)
INCLUDE (employee_id, first_name, last_name, hire_date)
WHERE status = 'ACTIVE'; VACUUM ANALYZE hr.employees;   -- Index Only Scan 유도를 위해
VACUUM 포함
EXPLAIN (ANALYZE, BUFFERS)
SELECT employee_id, first_name, last_name, hire_date, salary FROM hr.employees WHERE hire_date >= CURRENT_DATE -
INTERVAL '365 days' AND status = 'ACTIVE' ORDER BY salary DESC LIMIT 100;
```

5. 추가 포인트 및 조별 의견

- 복합 인덱스를 통한 조회로 시간 감축 확인
- 이미 데이터가 정렬된 경우. 100개 limit로 조회 시간 단축할 수 있다.

6. 결론 및 최적의 튜닝 Point

- 인덱스 설정이 순번대로 된 경우, 데이터 로드를 limit으로 제한하여 조회하면 빠르다.
- 만약, 인덱스 설정이 순번이 아닌 경우, 복합 인덱스로 시간 감축가능하다.

8. 인덱스 이외의 튜닝 실제 적용

SELECT * 제거와 조회 컬럼 축소

실행 쿼리

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT employee_id, email, hire_date, salary
FROM hr.employees
WHERE hire_date >= CURRENT_DATE - INTERVAL '365 days'
  AND status = 'ACTIVE'
ORDER BY salary DESC
LIMIT 100;
```

실측 결과 및 판단

```
                                                  QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=0.29..48.35 rows=100 width=36) (actual time=0.024..0.640 rows=100 loops=1)
   Buffers: shared hit=483
   ->  Index Scan using idx_employees_active_salary on employees  (cost=0.29..4501.82 rows=9367 width=36) (actual
time=0.023..0.631 rows=100 loops=1)
         Filter: (hire_date >= (CURRENT_DATE - '365 days'::interval))
         Rows Removed by Filter: 383
         Buffers: shared hit=483
 Planning Time: 0.145 ms
 Execution Time: 0.658 ms
(8 rows)
```

최종 튜닝 Point

화면에 필요한 컬럼만 조회하는 것은 유효한 비인덱스 튜닝이다. 다만 핵심 병목인 스캔과 정렬을 없애지는 못한다.

5. OR 조건 쿼리 튜닝

1. 문제

OR 조건 쿼리 작성 후 쿼리 튜닝 (조건 : 부서코드가 10 또는 직무가 3, 4, 5 안에 있는 사람 검색)

2. 튜닝 전 쿼리 및 출력결과

튜닝 전 테스트 5-1

쿼리문 (모두)

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM hr.employees
WHERE department_id = 10
  OR job_id IN (3,4,5);
```

출력결과

```
                                                    QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on employees  (cost=61.76..913.98 rows=4056 width=94) (actual time=0.291..2.161 rows=4085 loops=1)
   Recheck Cond: ((department_id = 10) OR (job_id = ANY ('{3,4,5}'::integer[])))
   Heap Blocks: exact=783
   Buffers: shared hit=790
   ->  BitmapOr  (cost=61.76..61.76 rows=4075 width=0) (actual time=0.211..0.212 rows=0 loops=1)
         Buffers: shared hit=7
         ->  Bitmap Index Scan on idx_employees_department_id  (cost=0.00..6.13 rows=245 width=0) (actual time=0.044..0.044
rows=251 loops=1)
               Index Cond: (department_id = 10)
               Buffers: shared hit=2
         ->  Bitmap Index Scan on idx_employees_job_id  (cost=0.00..53.60 rows=3830 width=0) (actual time=0.167..0.167
rows=3855 loops=1)
               Index Cond: (job_id = ANY ('{3,4,5}'::integer[]))
               Buffers: shared hit=5
 Planning Time: 0.119 ms
 Execution Time: 2.510 ms
(14 rows)
```

튜닝 전 테스트 5-2

쿼리문 (승우)

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT count(*) FROM hr.employees
WHERE department_id = 10
  OR job_id IN (3,4,5);
```

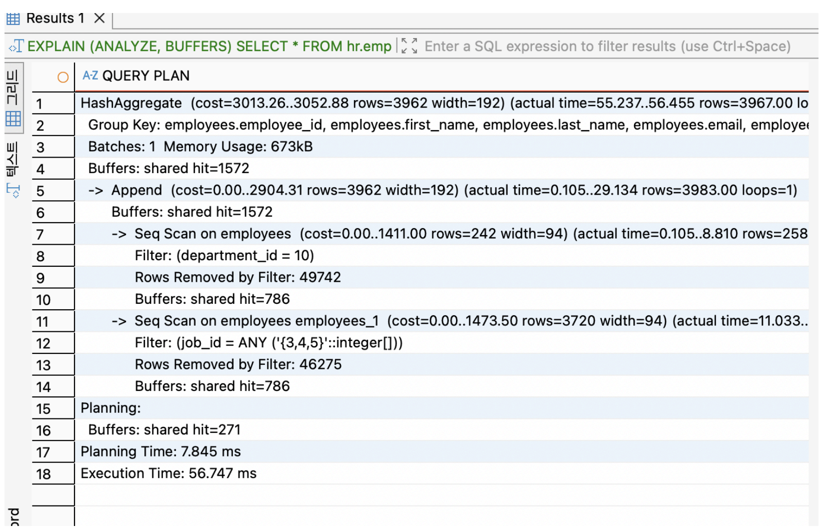
출력결과

```
                                                          QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------
 Aggregate  (cost=924.12..924.13 rows=1 width=8) (actual time=4.712..4.719 rows=1 loops=1)
   Buffers: shared hit=790
   ->  Bitmap Heap Scan on employees  (cost=61.76..913.98 rows=4056 width=0) (actual time=0.549..4.331 rows=4085 loops=1)
         Recheck Cond: ((department_id = 10) OR (job_id = ANY ('{3,4,5}'::integer[])))
        Heap Blocks: exact=783
         Buffers: shared hit=790
         ->  BitmapOr  (cost=61.76..61.76 rows=4075 width=0) (actual time=0.373..0.373 rows=0 loops=1)
               Buffers: shared hit=7
               ->  Bitmap Index Scan on idx_employees_department_id  (cost=0.00..6.13 rows=245 width=0) (actual
time=0.040..0.040 rows=251 loops=1)
                     Index Cond: (department_id = 10)
                     Buffers: shared hit=2
               ->  Bitmap Index Scan on idx_employees_job_id  (cost=0.00..53.60 rows=3830 width=0) (actual time=0.332..0.332
rows=3855 loops=1)
                     Index Cond: (job_id = ANY ('{3,4,5}'::integer[]))
                     Buffers: shared hit=5
 Planning:
   Buffers: shared hit=217
 Planning Time: 1.858 ms
 Execution Time: 4.784 ms
(18 rows)
```

쿼리문 (union 병합)

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM hr.employees WHERE department_id = 10
UNION
SELECT * FROM hr.employees WHERE job_id IN (3, 4, 5);
```

출력 결과


Results 1

EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM hr.emp | Enter a SQL expression to filter results (use Ctrl+Space)

	QUERY PLAN
1	HashAggregate (cost=3013.26..3052.88 rows=3962 width=192) (actual time=55.237..56.455 rows=3967.00 lo
2	Group Key: employees.employee_id, employees.first_name, employees.last_name, employees.email, employe
3	Batches: 1 Memory Usage: 673kB
4	Buffers: shared hit=1572
5	-> Append (cost=0.00..2904.31 rows=3962 width=192) (actual time=0.105..29.134 rows=3983.00 loops=1)
6	Buffers: shared hit=1572
7	-> Seq Scan on employees (cost=0.00..1411.00 rows=242 width=94) (actual time=0.105..8.810 rows=258
8	Filter: (department_id = 10)
9	Rows Removed by Filter: 49742
10	Buffers: shared hit=786
11	-> Seq Scan on employees employees_1 (cost=0.00..1473.50 rows=3720 width=94) (actual time=11.033..
12	Filter: (job_id = ANY ('{3,4,5}'::integer[]))
13	Rows Removed by Filter: 46275
14	Buffers: shared hit=786
15	Planning:
16	Buffers: shared hit=271
17	Planning Time: 7.845 ms
18	Execution Time: 56.747 ms

3. 튜닝 작업 내역

```
CREATE INDEX idx_employees_department_id
ON hr.employees (department_id);

CREATE INDEX idx_employees_job_id
ON hr.employees (job_id);

ANALYZE hr.employees;
```

4. 튜닝 후 쿼리 및 출력결과

튜닝 후 테스트 5-1

쿼리문

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM hr.employees
WHERE department_id = 10
   OR job_id IN (3,4,5);
```

출력결과

```
                                                                QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on employees  (cost=62.93..917.53 rows=4203 width=94) (actual time=0.241..1.752 rows=4085 loops=1)
   Recheck Cond: ((department_id = 10) OR (job_id = ANY ('{3,4,5}'::integer[])))
   Heap Blocks: exact=783
   Buffers: shared hit=790
   ->  BitmapOr  (cost=62.93..62.93 rows=4222 width=0) (actual time=0.168..0.169 rows=0 loops=1)
         Buffers: shared hit=7
         ->  Bitmap Index Scan on idx_employees_department_id  (cost=0.00..6.12 rows=244 width=0) (actual time=0.024..0.024
rows=251 loops=1)
               Index Cond: (department_id = 10)
               Buffers: shared hit=2
         ->  Bitmap Index Scan on idx_employees_job_id  (cost=0.00..54.70 rows=3978 width=0) (actual time=0.143..0.144
rows=3855 loops=1)
               Index Cond: (job_id = ANY ('{3,4,5}'::integer[]))
               Buffers: shared hit=5
 Planning Time: 0.145 ms
 Execution Time: 1.907 ms
(14 rows)
```

튜닝 후 테스트 5-2

쿼리문

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM hr.employees WHERE department_id = 10
UNION
SELECT * FROM hr.employees WHERE job_id IN (3,4,5);
```

출력결과

```
QUERY PLAN

-------------------------------------------------------------------------------------------------------------------------------------------------------------

HashAggregate (cost=1560.54..1602.76 rows=4222 width=192) (actual time=4.459..5.401 rows=4085 loops=1)

Group Key: employees.employee_id, employees.first_name, employees.last_name, employees.email, employees.phone,
employees.hire_date, employees.salary, employe
```

```
es.manager_id, employees.department_id, employees.job_id, employees.status

Batches: 1 Memory Usage: 985kB

Buffers: shared hit=1013

-> Append (cost=6.18..1444.43 rows=4222 width=192) (actual time=0.100..2.008 rows=4106 loops=1)

Buffers: shared hit=1013

-> Bitmap Heap Scan on employees (cost=6.18..526.93 rows=244 width=94) (actual time=0.099..0.487 rows=251 loops=1)

Heap Blocks: exact=223

Buffers: shared hit=225

-> Bitmap Index Scan on idx_employees_department_id (cost=0.00..6.12 rows=244 width=0) (actual time=0.041..0.041
rows=251 loops=1)

Index Cond: (department_id = 10)

Buffers: shared hit=2

-> Bitmap Heap Scan on employees employees_1 (cost=55.70..896.40 rows=3978 width=94) (actual time=0.230..1.304
rows=3855 loops=1)

Recheck Cond: (job_id = ANY ('{3,4,5}'::integer[]))

Heap Blocks: exact=783

Buffers: shared hit=788

-> Bitmap Index Scan on idx_employees_job_id (cost=0.00..54.70 rows=3978 width=0) (actual time=0.164..0.165 rows=3855
loops=1)

Index Cond: (job_id = ANY ('{3,4,5}'::integer[]))

Buffers: shared hit=5

Planning:

Buffers: shared hit=23

Planning Time: 3.358 ms

Execution Time: 5.636 ms

(24 rows)
```

5. 추가 포인트 및 조별 의견

- 인덱스 생성 (복합 및 개별 인덱스 각각)하여 조회하면 빠르다.

7. 결론 및 최적의 튜닝 Point

- 인덱스 생성 (복합 및 개별 인덱스 각각)하여 조회하면 빠르다.

8. 인덱스 이외의 튜닝 실제 적용

IN 단순화와 상호 배타 UNION ALL

실행 쿼리

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM hr.employees
WHERE department_id = 10 OR job_id BETWEEN 3 AND 5;

EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM hr.employees WHERE department_id = 10
UNION ALL
SELECT * FROM hr.employees
WHERE job_id IN (3,4,5) AND department_id <> 10;
```

실측 결과 및 판단

```
QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on employees  (cost=64.29..924.18 rows=4203 width=94) (actual time=0.531..6.628 rows=4085 loops=1)
   Recheck Cond: ((department_id = 10) OR ((job_id >= 3) AND (job_id <= 5)))
   Heap Blocks: exact=783
   Buffers: shared hit=790
   ->  BitmapOr  (cost=64.29..64.29 rows=4222 width=0) (actual time=0.363..0.364 rows=0 loops=1)
         Buffers: shared hit=7
         ->  Bitmap Index Scan on idx_employees_department_id  (cost=0.00..6.12 rows=244 width=0) (actual time=0.037..0.038
rows=251 loops=1)
               Index Cond: (department_id = 10)
               Buffers: shared hit=2
         ->  Bitmap Index Scan on idx_employees_job_id  (cost=0.00..56.07 rows=3978 width=0) (actual time=0.324..0.324
rows=3855 loops=1)
               Index Cond: ((job_id >= 3) AND (job_id <= 5))
               Buffers: shared hit=5
 Planning:
   Buffers: shared hit=207
 Planning Time: 1.873 ms
 Execution Time: 6.988 ms
(16 rows)
```

최종 튜닝 Point

조건식 재작성은 결과 집합이 동일한지 먼저 확인해야 한다. 현재 데이터에서는 원본 OR + BitmapOr가 가장 안정적이다.

종합 결론

2번. 이메일은 lower(email) 표현식 인덱스, status는 입력값을 정규화하고 status = 'INACTIVE'로 직접 비교한다.

3번. reverse(email) text_pattern_ops 인덱스를 생성하고 reverse(email) LIKE reverse('접미사') || '%'로 재작성한다.

4번. ORDER BY salary DESC + LIMIT 100을 먼저 지원하는 ACTIVE partial index로 정렬을 제거한다. 최근 입사자 비율이 크게 바뀌면 인덱스 설계를 재측정한다.

5번. department_id와 job_id에 각각 단일 인덱스를 생성하고 원본 OR 쿼리를 유지한다. 현재 분포에서 BitmapOr가 UNION보다 빠르고 의미도 명확하다.

실행 시간은 캐시와 동시 부하에 따라 달라질 수 있으므로, 실행 시간과 함께 Seq Scan, Sort, Rows Removed by Filter, Buffers 변화를 판단해야한다.